

Etcd + Raft +

Diving into etcd/raft



Daniel Chia

Follow

3 min read · Nov 5, 2017



82



3



...

As part of implementing Raft for DDB, I've been reading the Raft implementation in etcd.

Why did I choose etcd? etcd is written in Go, and it was interesting to see how they structured their code, especially w.r.t to concurrency. etcd also forms the backing store for Kubernetes, and its Raft implementation is used in CockroachDB, so they must be doing something right! The authors claim it is the most widely used Raft library in production.

At work we often give *Life of a Request* style presentations, and I find them incredibly helpful to understand both and micro and macro levels how a system works. That's what I'll try to do here.

If you're not familiar with Raft, reviewing the paper may potentially be helpful.

Overview

The raft implementation lives in `etcd/raft`, with majority of the heavy lifting happening in a few files:

- `raft.go` As you might guess from the name, this is heart of the implementation. `raft.raft` exposes a finite state machine, and is **not** thread-safe.
- `node.go` `raft.Node` wraps `raft.raft` takes care of synchronization and some of the plumbing.
- `raftpb/raft.proto` Proto definitions.

Application $\Leftrightarrow$ Node Loop

As stated in the README, one should service `raft.Node` like so, most likely in a goroutine.

```

1  for {
2      select {
3      case <-s.Ticker:
4          n.Tick()
5      case rd := <-s.Node.Ready():
6          saveToStorage(rd.HardState, rd.Entries, rd.Snapshot)
7          send(rd.Messages)
8          if !raft.IsEmptySnap(rd.Snapshot) {
9              processSnapshot(rd.Snapshot)
10         }
11         for _, entry := range rd.CommittedEntries {
12             process(entry)
13         }
14         s.Node.Advance()
15     case <-s.done:
16         return
17     }
18 }

```

application_node_loop.go hosted with ❤ by GitHub

[view raw](#)

Servicing raft.Node, some simplifications applied.

Let's dive into what this loop is doing:

Ticking(`case <-s.Ticker`)

This supplies ticks, basically a notion of time, to the node. Certain actions like heartbeats and election timeouts are triggered by the passing of time.

Externalizing raft.Node Events(`case rd := <-s.Node.Ready()`)

`raft.Node` is not responsible for saving various persistent state to stable storage (e.g. disk), sending messages (RPCs) to other raft nodes, and applying any newly committed log entries. That's what this particular case handles. I like to think of this step as externalizing changes from `raft.Node`.

The channel returned by `Node.Ready()` receives a `raft.Ready`, when `raft.Node` needs any of the above performed. The `raft.Ready` struct contains a copy of state that needs to be externalized.

Once all the necessary actions have been performed, invoking `Node.Advance()` lets the node know the application is ready for another batch of changes.

I thought it was pretty interesting to see etcd exploiting concurrency here — while the application $\Leftrightarrow$ node loop is potentially blocked / busy writing state to stable storage and sending RPCs, `raft.Node` concurrently continues to process any new proposals and raft messages.

raft.Node

As mentioned earlier, `raft.raft` which implements the core Raft logic isn't threadsafe, and `raft.Node` is sort of the glue between the public threadsafe API and `raft.raft`. Not surprisingly, this is orchestrated using a handful of channels serviced by an event loop of sorts, plumbing values to and from channels.

Most of the interesting behavior is covered above. What really stood out on first reading was the `Ready()` and `Advance()` dance. I wonder why not push the loop logic into the internals of `raft.Node` rather than force applications to have to do with it.

raft.Progress

`raft.Progress` keeps track of the state of replication per follower.

It's purpose is fully documented in [design.md](#). Summarized:

- Progress limits the maximum number of messages in flight to reduce wasted work and bandwidth. The actual limit depends on the state of `Progress`.
- `Progress` initially starts out in a conservation 'probe' state, where it sends one RPC at a time to the follower. In this state, the leader isn't sure how far the follower is caught up.
- Once the leader is certain of the follower's log state, it moves to a `replicate` state for `Progress`, where new log entries append RPCs are sent in a pipelined fashion. Appends are sent optimistically even when a previous RPC is still outstanding. When everything is operating normally, this reduces latency as we do not have to wait for previous append RPCs to finish.

Life of a Write

This assumes the request was made to the current raft leader.

1. A write starts with `node.Propose`.
2. This eventually causes `raft.StepLeader` to be invoked.
3. The leader queues the proposal for storage to its own log.
4. The leader updates all `Progress`, indicating that a new entry has been appended.
5. For each follower, based on the state of `Progress` for the follower, the leader queues appropriate append entry RPCs.
6. When the leader realizes that enough followers have accepted the log, it commits the log entry and applies the log entry.

Etcd +

Raft +



Written by Daniel Chia

372 followers · 5 following

Follow

Software Engineer at Google. Opinions stated are my own, not of my company.



